UNIVERSITY OF PETROLEUM AND ENERGY STUDIES
Dehradun, Uttarakhand

School of Computer Science

Containerization and DevOps Project

Assignment 1:Containerized Web Application with PostgreSQL using Docker Compose and Ipvlan

Name: Rishu Raj
SAP ID: 500119484
Course: BTech CSE
Specialization: CCVT
Batch: 1

Contents

1 GitHub Repository **2**

2 Separate Dockerfiles **2**
2.1 Backend Dockerfile 2
2.2 Database Dockerfile 3

3 Docker Compose Configuration **3**
3.1 docker-compose.yml 3

4 Network Creation Command **4**

5 Screenshot Proofs **5**
5.1 Docker Network Inspect 5
5.2 Container IPs 6
5.3 Volume Persistence Test 9

6 Build Optimization Explanation **10**
6.1 Backend Multi-Stage Dockerfile 11

7 Network Design Diagram **11**

8 Image Size Comparison **12**

9 macvlan vs ipvlan Comparison **14**

1 GitHub Repository

The complete project source code is available in the GitHub repository.

Repository Link:
Containerized Web Application Project: https://github.com/rajrishu1401/ContainerizationAndDevOP

Github Page Link:
Github Pages Link: https://rajrishu1401.github.io/ContainerizationAndDevOPsLab/

Repository includes:

- Backend source code
- Database configuration
- Frontend UI
- Dockerfiles
- Docker Compose configuration
- Documentation

2 Separate Dockerfiles

The project contains separate Dockerfiles for each service.

2.1 Backend Dockerfile

Location:

```
backend/Dockerfile
```

Features:

- Multi-stage build
- Alpine base image
- Non-root user
- Minimal runtime environment

2.2 Database Dockerfile

Location:

```
database/Dockerfile
```

Environment variables configured:

```
POSTGRES_DB
POSTGRES_USER
POSTGRES_PASSWORD
```

3 Docker Compose Configuration

Docker Compose orchestrates the entire container stack.

Services defined:

- frontend
- backend
- database

Configuration includes:

- Static IP assignment
- External IPVLAN network
- Named volumes
- Environment variables
- Restart policy

3.1 docker-compose.yml

```
version: "3.9"

services:

  database:
    build: ./database
    container_name: postgres_db
    restart: always
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      mynet:
```

```
        ipv4_address: 172.24.10.10

  backend:
    build: ./backend
    container_name: backend_api
    restart: always
    depends_on:
      - database
    environment:
      DB_HOST: 172.24.10.10
      DB_USER: appuser
      DB_PASSWORD: securepassword
      DB_NAME: appdb
    networks:
      mynet:
        ipv4_address: 172.24.10.20

  frontend:
    build: ./frontend
    container_name: frontend_ui
    restart: always
    networks:
      mynet:
        ipv4_address: 172.24.10.30

volumes:
  postgres_data:

networks:
  mynet:
    external: true
```

4 Network Creation Command

The IPVLAN network is created manually using the following command:

```
docker network create -d ipvlan \
--subnet=172.24.0.0/16 \
--gateway=172.24.0.1 \
-o parent=eth0 \
mynet
```

5 Screenshot Proofs

This section provides proof that the containerized application was successfully deployed using Docker Compose with an IPVLAN network and persistent storage.

5.1 Docker Network Inspect

To verify the created network and connected containers, the following command was executed:

```
docker network inspect mynet
```

This command shows the network configuration including:

- Network driver (ipvlan)
- Subnet configuration
- Gateway address
- Connected containers
- Assigned container IP addresses

```
raj_rishu@LAPTOP-IQ7I38PQ:/mnt/d/Courses/BTech/6th_Sem/Containerization_and_DevOps/Assignment1/OptimizedSize$ docker network inspect mynet
[
    {
        "Name": "mynet",
        "Id": "752d63b2a51fcdd83660846e1f19c8bdf04cd28ba7029f1e7a9ec024e3582db3",
        "Created": "2026-03-15T19:52:34.016919798Z",
        "Scope": "local",
        "Driver": "ipvlan",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.24.0.0/16",
                    "Gateway": "172.24.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "03913ec501173872fa417b3478bc8a94b1e3d56d64a1997a444f43ef33fa7744": {
                "Name": "postgres_db",
                "EndpointID": "0b0d8a5f452fc6e4130a65a66ef80129a39020266dd8c78cbf721dbdc4ed175e",
                "MacAddress": "",
                "IPv4Address": "172.24.10.10/16",
                "IPv6Address": ""
            },
            "288a3a93a29224e0ce745570c1335d13f1e9d815fb634a612176123a9d4a0c12": {
                "Name": "frontend_ui",
                "EndpointID": "2ddf5c1a66c7cf2b4649a266a0e91e6a7d841458cbc7f457df0fd6957acc1f06",
                "MacAddress": "",
                "IPv4Address": "172.24.10.30/16",
                "IPv6Address": ""
            },
            "685353c073f997e9d104e643f1e3f3d650bf53300e38c517a4d32ecbf3509137": {
                "Name": "backend_api",
                "EndpointID": "6bd0c32f92a004b7f7fe80353510af30499a788b685fd2d5147111047a28a7a0",
                "MacAddress": "",
                "IPv4Address": "172.24.10.20/16",
                "IPv6Address": ""
            }
        },
```

Figure 1: Docker Network Inspect Output

5.2 Container IPs

Each container in the Docker Compose setup was assigned a static IP address. The following commands were used to verify the IP addresses of the containers.

```
docker inspect backend_api
docker inspect frontend_ui
docker inspect postgres_db
```

The inspect command displays detailed information about the container including the assigned IP address within the IPVLAN network.

```
    "NetworkSettings": {
        "Bridge": "",
        "SandboxID": "f9b9a6da98f7292c3cbc533afa75ef0e262ae920842193f702f95f9c696cabd4",
        "SandboxKey": "/var/run/docker/netns/f9b9a6da98f7",
        "Ports": {},
        "HairpinMode": false,
        "LinkLocalIPv6Address": "",
        "LinkLocalIPv6PrefixLen": 0,
        "SecondaryIPAddresses": null,
        "SecondaryIPv6Addresses": null,
        "EndpointID": "",
        "Gateway": "",
        "GlobalIPv6Address": "",
        "GlobalIPv6PrefixLen": 0,
        "IPAddress": "",
        "IPPrefixLen": 0,
        "IPv6Gateway": "",
        "MacAddress": "",
        "Networks": {
            "mynet": {
                "IPAMConfig": {
                    "IPv4Address": "172.24.10.20"
                },
                "Links": null,
                "Aliases": [
                    "backend",
                    "685353c073f9"
                ],
                "MacAddress": "",
                "DriverOpts": null,
                "GwPriority": 0,
                "NetworkID": "752d63b2a51fcdd83660846e1f19c8bdf04cd28ba7029f1e7a9ec024e3582db3",
                "EndpointID": "6bd0c32f92a004b7f7fe80353510af30499a788b685fd2d5147111047a28a7a0",
                "Gateway": "172.24.0.1",
                "IPAddress": "172.24.10.20",
                "IPPrefixLen": 16,
                "IPv6Gateway": "",
                "GlobalIPv6Address": "",
                "GlobalIPv6PrefixLen": 0,
                "DNSNames": [
                    "backend_api",
                    "backend",
                    "685353c073f9"
                ]
            }
        }
    }
}
```


Figure 2: Backend Container IP Address

```
        "NetworkSettings": {
            "Bridge": "",
            "SandboxID": "5a6d3bf67f3a0cd83153cb3b4d7696bd9685176b078499cfa812e45ecfdc147f",
            "SandboxKey": "/var/run/docker/netns/5a6d3bf67f3a",
            "Ports": {},
            "HairpinMode": false,
            "LinkLocalIPv6Address": "",
            "LinkLocalIPv6PrefixLen": 0,
            "SecondaryIPAddresses": null,
            "SecondaryIPv6Addresses": null,
            "EndpointID": "",
            "Gateway": "",
            "GlobalIPv6Address": "",
            "GlobalIPv6PrefixLen": 0,
            "IPAddress": "",
            "IPPrefixLen": 0,
            "IPv6Gateway": "",
            "MacAddress": "",
            "Networks": {
                "mynet": {
                    "IPAMConfig": {
                        "IPv4Address": "172.24.10.30"
                    },
                    "Links": null,
                    "Aliases": [
                        "288a3a93a292",
                        "frontend"
                    ],
                    "MacAddress": "",
                    "DriverOpts": null,
                    "GwPriority": 0,
                    "NetworkID": "752d63b2a51fcdd83660846e1f19c8bdf04cd28ba7029f1e7a9ec024e3582db3",
                    "EndpointID": "2ddf5c1a66c7cf2b4649a266a0e91e6a7d841458cbc7f457df0fd6957acc1f06",
                    "Gateway": "172.24.0.1",
                    "IPAddress": "172.24.10.30",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "DNSNames": [
                        "frontend_ui",
                        "288a3a93a292",
                        "frontend"
                    ]
                }
            }
        }
    }
]
```


Figure 3: Frontend Container IP Address

```
        "NetworkSettings": {
            "Bridge": "",
            "SandboxID": "50060ba47d35d11c76dc571317fef224a83af1da1f7be1b56f8bc2f82bc7cffd",
            "SandboxKey": "/var/run/docker/netns/50060ba47d35",
            "Ports": {},
            "HairpinMode": false,
            "LinkLocalIPv6Address": "",
            "LinkLocalIPv6PrefixLen": 0,
            "SecondaryIPAddresses": null,
            "SecondaryIPv6Addresses": null,
            "EndpointID": "",
            "Gateway": "",
            "GlobalIPv6Address": "",
            "GlobalIPv6PrefixLen": 0,
            "IPAddress": "",
            "IPPrefixLen": 0,
            "IPv6Gateway": "",
            "MacAddress": "",
            "Networks": {
                "mynet": {
                    "IPAMConfig": {
                        "IPv4Address": "172.24.10.10"
                    },
                    "Links": null,
                    "Aliases": [
                        "03913ec50117",
                        "database"
                    ],
                    "MacAddress": "",
                    "DriverOpts": null,
                    "GwPriority": 0,
                    "NetworkID": "752d63b2a51fcdd83660846e1f19c8bdf04cd28ba7029f1e7a9ec024e3582db3",
                    "EndpointID": "0b0d8a5f452fc6e4130a65a66ef80129a39020266dd8c78cbf721dbdc4ed175e",
                    "Gateway": "172.24.0.1",
                    "IPAddress": "172.24.10.10",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "DNSNames": [
                        "postgres_db",
                        "03913ec50117",
                        "database"
                    ]
                }
            }
        }
    }
```


Figure 4: Database Container IP Address

5.3 Volume Persistence Test

Persistent storage was implemented using a Docker named volume. This ensures that the database data remains available even if containers are stopped or recreated.

First, data was inserted into the application using the frontend interface.

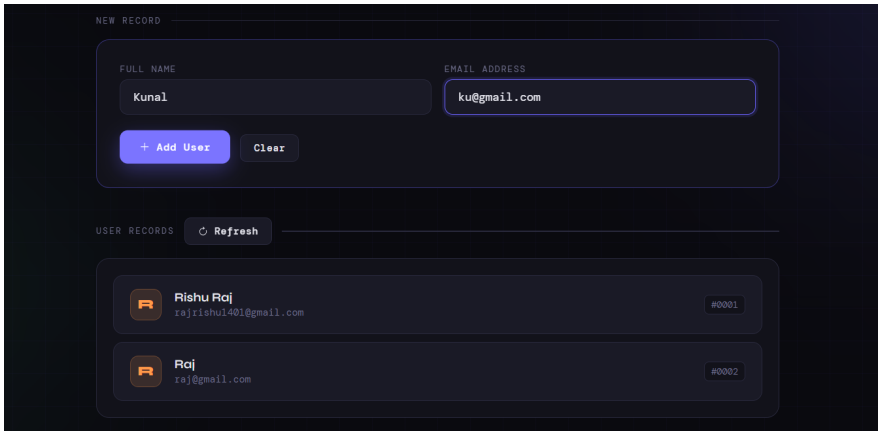

Figure 5: Data inserted into the application

Next, the containers were stopped using the following command:

```
docker compose down
```

This command stops and removes the containers but does not delete the persistent volume.

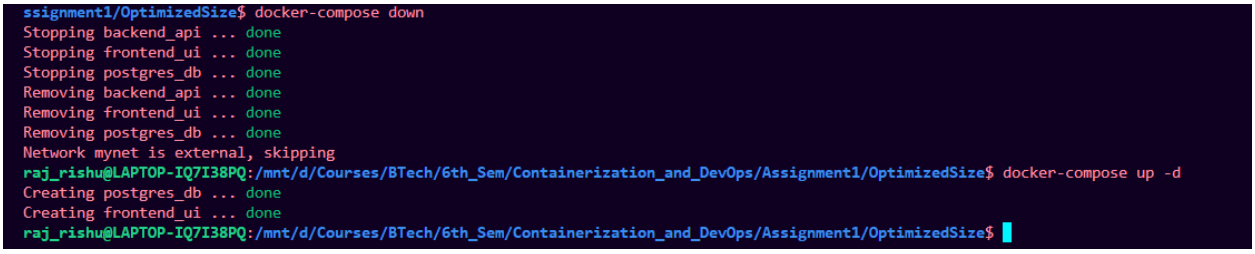

Figure 6: Containers stopped

The containers were then restarted using:

```
docker compose up -d
```

After restarting the containers, the previously inserted data was still available, confirming that the PostgreSQL database was using persistent storage through Docker volumes.

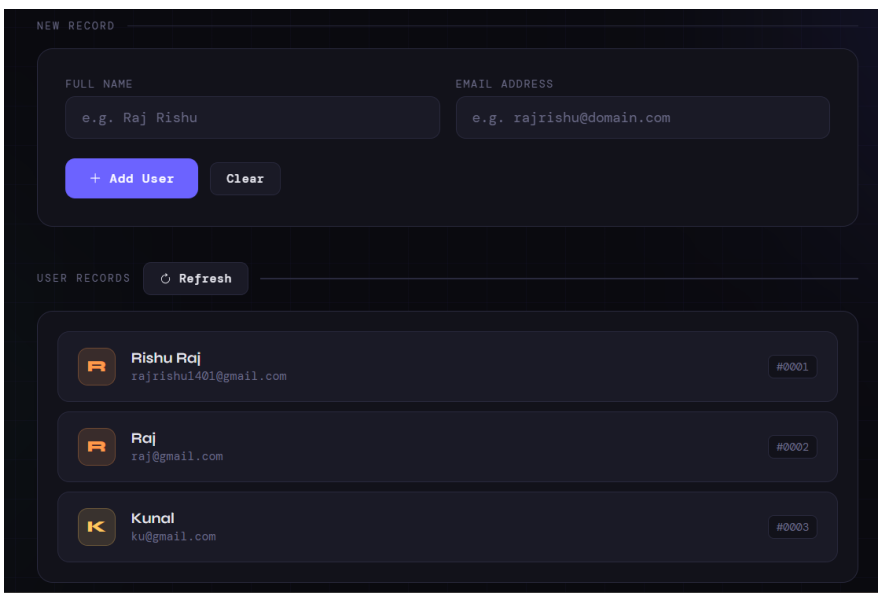

Figure 7: Data persists after container restart

6 Build Optimization Explanation

The backend container was optimized using a multi-stage Docker build and Alpine Linux base image. Multi-stage builds help reduce the final container image size by separating the build environment from the runtime environment.

6.1 Backend Multi-Stage Dockerfile

The following Dockerfile was used to build the backend container:

```
FROM node:20-alpine AS builder

WORKDIR /app
COPY package.json .
RUN npm install --production
COPY server.js .

FROM node:20-alpine

WORKDIR /app
COPY --from=builder /app /app

RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 5000
CMD ["node", "server.js"]
```

In this Dockerfile:

- The **builder stage** installs dependencies and prepares the application.
- The **runtime stage** copies only the required files from the builder stage.
- A **non-root user** is created to improve container security.
- The **Alpine Linux image** keeps the container lightweight.

These optimizations significantly reduce image size and improve container efficiency.

7 Network Design Diagram

The application containers communicate using an IPVLAN network. Each service is assigned a static IP address within the subnet.

Container IP configuration:

- PostgreSQL Container – 172.24.10.10
- Backend Container – 172.24.10.20
- Frontend Container – 172.24.10.30

The network was created using the following command:

```
docker network create -d ipvlan \
--subnet=172.24.0.0/16 \
--gateway=172.24.0.1 \
-o parent=eth0 \
mynet
```

The architecture of the system is shown in the diagram below.

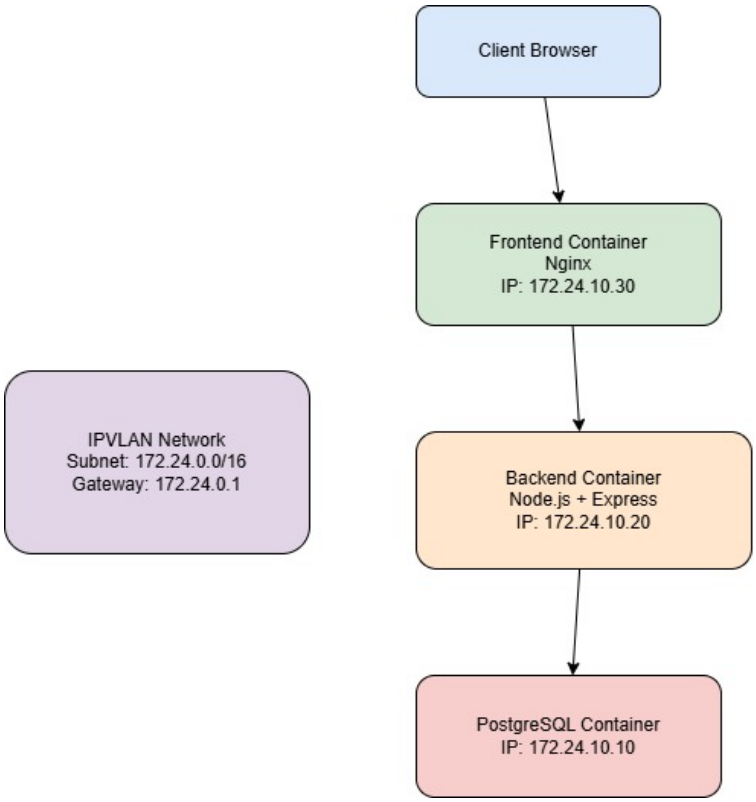

Figure 8: IPVLAN Network Architecture

8 Image Size Comparison

To demonstrate Docker image optimization, two backend builds were created:

- Optimized build using Alpine Linux and multi-stage Dockerfile
- Normal build using standard Node.js image without multi-stage build

The following command was used to check Docker image sizes:

```
docker images
```

```
raj_rishu@LAPTOP-IQ7I38PQ:/mnt/d/Courses/BTech/6th_Sem/Containerization_and_DevOps/Assignment1/Normal$ docker images
REPOSITORY                TAG         IMAGE ID       CREATED          SIZE
normal_backend            latest      3aa9f899ebce   51 seconds ago   1.1GB
optimizedsize_frontend    latest      4146c82ea210   5 minutes ago    62.2MB
normal_frontend           latest      1e71fe59ff64   29 minutes ago   62.2MB
optimizedsize_backend     latest      59b82a549567   30 minutes ago   138MB
<none>                    <none>      be17a7c458ca   30 minutes ago   145MB
optimizedsize_database    latest      d1c59b934d1e   30 minutes ago   276MB
normal_database           latest      d1c59b934d1e   30 minutes ago   276MB
```

Figure 9: Docker Image Size Comparison

From the comparison, it can be observed that the optimized image built using Alpine and multi-stage build is significantly smaller than the normal image.

Smaller images provide advantages such as faster image downloads, reduced storage usage, and quicker container startup time.

9 macvlan vs ipvlan Comparison

Feature	**Macvlan**	**Ipvlan**
Network Layer	Operates at Layer 2 of the OSI model and assigns each container a unique MAC address.	Can operate in Layer 2 or Layer 3 mode and uses the host MAC address while assigning unique IP addresses to containers.
MAC Address	Each container gets its own unique MAC address, appearing as a separate physical device on the network.	Containers share the host's MAC address but are differentiated using unique IP addresses.
Host Communication	Containers cannot communicate directly with the host due to macvlan network isolation. Special configuration is required to enable host-container communication.	Host communication is generally easier since containers share the host network interface.
Network Isolation	Provides strong isolation between containers and the host network.	Provides moderate isolation while allowing better host interaction.
Performance	Very efficient because containers communicate directly with the physical network interface.	Similar performance with reduced overhead in certain virtualized environments.
Scalability	Suitable when containers must appear as independent devices on the physical LAN.	Better suited for environments with many containers because it avoids MAC address exhaustion.
Typical Use Case	Useful when containers must behave like physical machines on the network (e.g., network appliances).	Commonly used in virtualized environments, cloud systems, or environments like WSL where macvlan limitations exist.

In this project, the IPVLAN network driver was selected because it works reliably in virtualized environments such as WSL2. IPVLAN allows containers to communicate with each other using static IP addresses while maintaining compatibility with the host networking environment.